# Visualizing Data using t-SNE

Laurens van der Maaten & Geoffrey Hinton (2008)

Iker Martin Canut

26 de noviembre de 2025

# Motivación

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.
- Técnicas no lineales previas (Sammon, CCA, SNE, Isomap, MVU, LLE):

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.
- Técnicas no lineales previas (Sammon, CCA, SNE, Isomap, MVU, LLE):
  - Funcionan bien

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.
- Técnicas no lineales previas (Sammon, CCA, SNE, Isomap, MVU, LLE):
  - Funcionan bien en datasets artificiales.

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.
- Técnicas no lineales previas (Sammon, CCA, SNE, Isomap, MVU, LLE):
  - Funcionan bien en datasets artificiales.
  - En datos reales no pueden retener las estructuras, ni locales o globales.

# Motivación

- Queremos **visualizar** y **preservar** la estructura de datos con muchas dimensiones.
- Técnicas lineales (PCA, MDS):
  - Solo pueden preservar separaciones en mapas lineales.
  - Cuando los datos viven en un espacio no lineal lo crucial que es mantener muy juntos a los puntos muy similares no se puede lograr.
- Técnicas no lineales previas (Sammon, CCA, SNE, Isomap, MVU, LLE):
  - Funcionan bien en datasets artificiales.
  - En datos reales no pueden retener las estructuras, ni locales o globales.
- **Stochastic Neighbor Embedding (SNE)**.

- La **similaridad** de $x_j$ a $x_i$

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino,

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino, si los vecinos se eligen en proporción a su densidad de probabilidad bajo una Gaussiana centrada en $x_i$:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino, si los vecinos se eligen en proporción a su densidad de probabilidad bajo una Gaussiana centrada en $x_i$:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

- En menos dimensiones (2D o 3D), definimos la similaridad $q_{j|i}$ sobre los puntos $y_i$, tomando la varianza como $1/\sqrt{2}$:

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\|y_i - y_k\|^2\right)}$$

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino, si los vecinos se eligen en proporción a su densidad de probabilidad bajo una Gaussiana centrada en $x_i$:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

- En menos dimensiones (2D o 3D), definimos la similaridad $q_{j|i}$ sobre los puntos $y_i$, tomando la varianza como $1/\sqrt{2}$:

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\|y_i - y_k\|^2\right)}$$

- Si los puntos del mapa $y_i$ y $y_j$ modelan correctamente la similaridad de los puntos originales $x_i$ y $x_j$, entonces $p_{j|i} = q_{j|i}$. Buscamos minimizar la diferencia.

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino, si los vecinos se eligen en proporción a su densidad de probabilidad bajo una Gaussiana centrada en $x_i$:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

- En menos dimensiones (2D o 3D), definimos la similaridad $q_{j|i}$ sobre los puntos $y_i$, tomando la varianza como $1/\sqrt{2}$:

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\|y_i - y_k\|^2\right)}$$

- Si los puntos del mapa $y_i$ y $y_j$ modelan correctamente la similaridad de los puntos originales $x_i$ y $x_j$, entonces $p_{j|i} = q_{j|i}$. Buscamos minimizar la diferencia.
- Trabajamos con la divergencia de Kullback–Leibler:

$$C = \sum_i \mathrm{KL}(P_i \parallel Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- La **similaridad** de $x_j$ a $x_i$ es una probabilidad condicional $p_{j|i}$ que mide qué tan probable es que $x_i$ elija a $x_j$ como vecino, si los vecinos se eligen en proporción a su densidad de probabilidad bajo una Gaussiana centrada en $x_i$:

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

- En menos dimensiones (2D o 3D), definimos la similaridad $q_{j|i}$ sobre los puntos $y_i$, tomando la varianza como $1/\sqrt{2}$:

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\|y_i - y_k\|^2\right)}$$

- Si los puntos del mapa $y_i$ y $y_j$ modelan correctamente la similaridad de los puntos originales $x_i$ y $x_j$, entonces $p_{j|i} = q_{j|i}$. Buscamos minimizar la diferencia.
- Trabajamos con la divergencia de Kullback–Leibler:

$$C = \sum_i \mathrm{KL}(P_i \parallel Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- Y buscamos minimizar esta suma usando un metodo de descenso de gradiente.

# Problemas de SNE

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

    - Costo no convexo (simulated annealing para solucionar).

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

    - Costo no convexo (simulated annealing para solucionar).
    - Tuning cuidadoso de parámetros.

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

    - Costo no convexo (simulated annealing para solucionar).
    - Tuning cuidadoso de parámetros.
- Cuarta complicación: **Crowding problem**:

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

    - Costo no convexo (simulated annealing para solucionar).
    - Tuning cuidadoso de parámetros.
- Cuarta complicación: **Crowding problem**:
  - En alta dimensión hay "mucho volumen" para puntos alejados.

# Problemas de SNE

- Una primera complicación sale porque la función de coste **no es simétrica**:
  - Penaliza menos los falsos vecinos (puntos cercanos en el mapa pero lejanos en alta dimensión) que los falsos lejanos (puntos lejanos en el mapa pero cercanos en alta dimensión).
  - Dificulta la optimización porque se enfoca en retener la estructura local.
- Segunda complicación: elección de la varianza $\sigma_i$ para cada punto $x_i$.
- Tercera complicación: Optimización:
  - Si bien el gradiente tiene una forma relativamente simple,

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})(y_i - y_j)$$

    - Costo no convexo (simulated annealing para solucionar).
    - Tuning cuidadoso de parámetros.
- Cuarta complicación: **Crowding problem**:
  - En alta dimensión hay "mucho volumen" para puntos alejados.
  - En 2D no hay suficiente espacio: los puntos se amontonan en el centro.

# t-SNE: cambios clave sobre SNE

# t-SNE: cambios clave sobre SNE

- **Versión simétrica de SNE**:

# t-SNE: cambios clave sobre SNE

- **Versión simétrica de SNE**:
- Definimos

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$q_{ij} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq l} \exp\left(-\|y_k - y_l\|^2\right)}$$

# t-SNE: cambios clave sobre SNE

- **Versión simétrica de SNE**:
- Definimos

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$q_{ij} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq l} \exp\left(-\|y_k - y_l\|^2\right)}$$

- La función de coste queda:

$$C = \mathrm{KL}(P \,\|\, Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

# t-SNE: cambios clave sobre SNE

- **Versión simétrica de SNE**:
- Definimos

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$q_{ij} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq l} \exp\left(-\|y_k - y_l\|^2\right)}$$

- La función de coste queda:

$$C = \mathrm{KL}(P \,\|\, Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

y tenemos la propiedad de que $p_{ij} = p_{ji}$ y $q_{ij} = q_{ji}$.

# t-SNE: cambios clave sobre SNE

- **Versión simétrica de SNE**:
- Definimos

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$q_{ij} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq l} \exp\left(-\|y_k - y_l\|^2\right)}$$

- La función de coste queda:

$$C = \mathrm{KL}(P \,\|\, Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

  y tenemos la propiedad de que $p_{ij} = p_{ji}$ y $q_{ij} = q_{ji}$.
- Y el gradiente queda:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)$$

# t-SNE: cambios clave sobre SNE (cont.)

# t-SNE: cambios clave sobre SNE (cont.)

- Pero todavia nos queda el **crowding problem**.

# t-SNE: cambios clave sobre SNE (cont.)

- Pero todavia nos queda el **crowding problem**.
- Solución: usar una **distribución heavy-tailed** en el espacio de baja dimensión.

# t-SNE: cambios clave sobre SNE (cont.)

- Pero todavia nos queda el **crowding problem**.
- Solución: usar una **distribución heavy-tailed** en el espacio de baja dimensión.
- En lugar de una Gaussiana, usamos una **Student-t con 1 grado de libertad**:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|y_k - y_l\|^2)^{-1}}$$

# Efecto de la Student-t en 2D

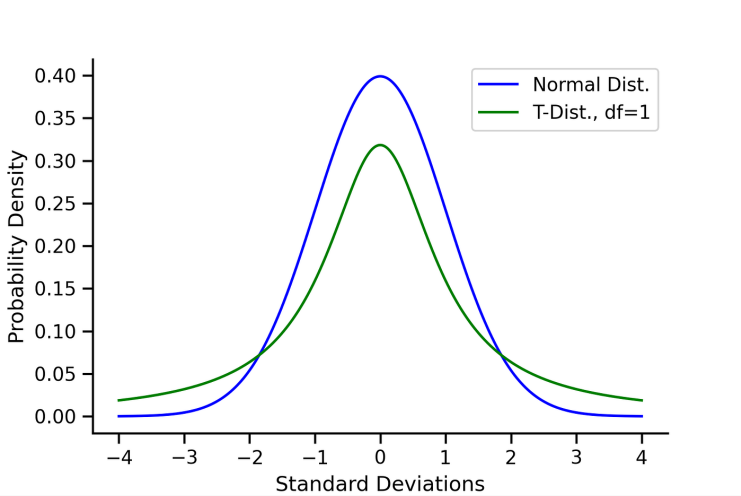



**Figura 1**: Comparación entre una distribución Gaussiana y una Student-t con 1 grado de libertad en 2D.

# Efecto de la Student-t en 2D

- Consecuencias:

# Efecto de la Student-t en 2D

- Consecuencias:
  - Se reduce el **crowding problem**.

# Efecto de la Student-t en 2D

- Consecuencias:
  - Se reduce el **crowding problem**.
  - Fuertes repulsiones solo donde hace falta: puntos disímiles demasiado cerca.

# Efecto de la Student-t en 2D

- Consecuencias:
  - Se reduce el **crowding problem**.
  - Fuertes repulsiones solo donde hace falta: puntos disímiles demasiado cerca.
- El gradiente termina quedando:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1}$$

# Algoritmo t-SNE

# Algoritmo t-SNE

1. Preprocesar con PCA (por ejemplo a 30 dimensiones).

# Algoritmo t-SNE

1. Preprocesar con PCA (por ejemplo a 30 dimensiones).
2. Calcular $p_{ij}$ usando los pares $p_{j|i}$

# Algoritmo t-SNE

1. Preprocesar con PCA (por ejemplo a 30 dimensiones).
2. Calcular $p_{ij}$ usando los pares $p_{j|i}$
3. Se empiezan los puntos $y_i$ muy cerca del origen: $\mathcal{Y}^{(0)} = \{y_1, y_2, ..., y_n\}$ de $\mathcal{N}(0, 10^{-4}I)$

# Algoritmo t-SNE

1. Preprocesar con PCA (por ejemplo a 30 dimensiones).
2. Calcular $p_{ij}$ usando los pares $p_{j|i}$
3. Se empiezan los puntos $y_i$ muy cerca del origen: $\mathcal{Y}^{(0)} = \{y_1, y_2, ..., y_n\}$ de $\mathcal{N}(0, 10^{-4}I)$
4. Iterar $t = 1$ hasta $T$:
   - 4.1 Calcular $q_{ij}$.
   - 4.2 Calcular el gradiente $\frac{\partial C}{\partial \mathcal{Y}}$.
   - 4.3 Actualizar

$$\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\partial C}{\partial \mathcal{Y}} + \alpha(t)(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)})$$

   donde $\eta$ es la tasa de aprendizaje y $\alpha(t)$ el momentum.

# Algoritmo t-SNE

1. Preprocesar con PCA (por ejemplo a 30 dimensiones).
2. Calcular $p_{ij}$ usando los pares $p_{j|i}$
3. Se empiezan los puntos $y_i$ muy cerca del origen: $\mathcal{Y}^{(0)} = \{y_1, y_2, ..., y_n\}$ de $\mathcal{N}(0, 10^{-4}I)$
4. Iterar $t = 1$ hasta $T$:
   - 4.1 Calcular $q_{ij}$.
   - 4.2 Calcular el gradiente $\frac{\partial C}{\partial \mathcal{Y}}$.
   - 4.3 Actualizar

$$\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\partial C}{\partial \mathcal{Y}} + \alpha(t)(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)})$$

   donde $\eta$ es la tasa de aprendizaje y $\alpha(t)$ el momentum.

Opcionalmente, usar **early exaggeration** multiplicando $p_{ij}$ por 4 en las primeras iteraciones para formar clusters más compactos que queden separados

# Experimentos

# Experimentos

- Comparar t-SNE con:

# Experimentos

- Comparar t-SNE con:
  - Sammon mapping.
  - Isomap.
  - Locally Linear Embedding (LLE).

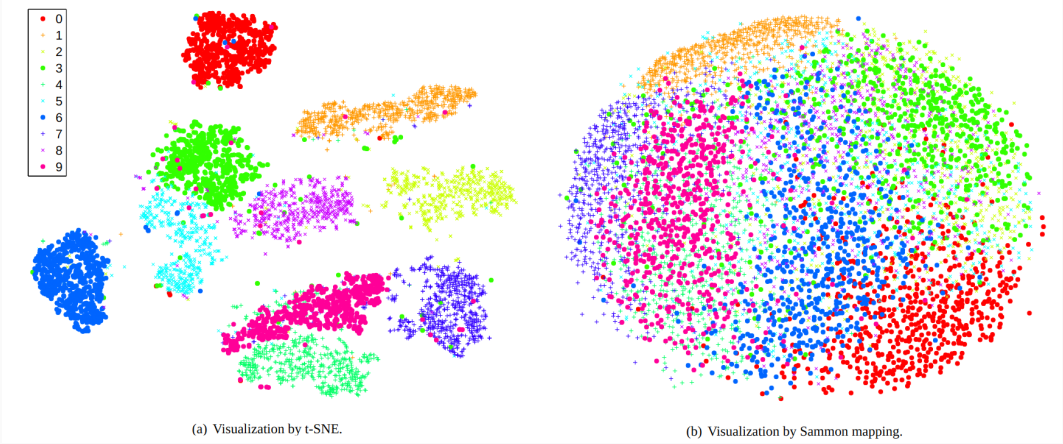
# Experimentos

- Comparar t-SNE con:
  - Sammon mapping.
  - Isomap.
  - Locally Linear Embedding (LLE).
- Datasets:

# Experimentos

- Comparar t-SNE con:
  - Sammon mapping.
  - Isomap.
  - Locally Linear Embedding (LLE).
- Datasets:
  - MNIST (60000 imágenes de dígitos escritos a mano).
  - Olivetti faces (400 imágenes de caras humanas).
  - COIL-20 (1440 imágenes de objetos rotados).

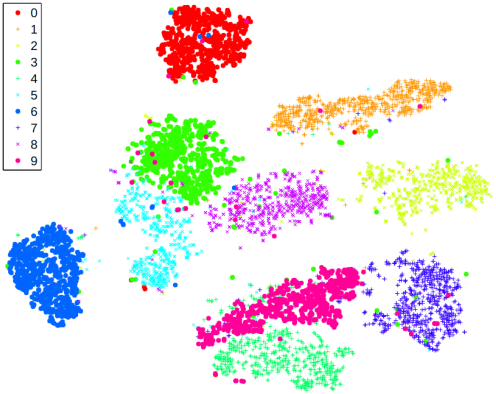
# Ejemplo: MNIST



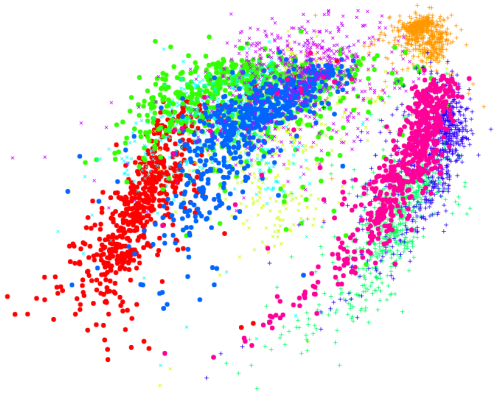

(a) Visualization by t-SNE.

(b) Visualization by Sammon mapping.

# Ejemplo: MNIST

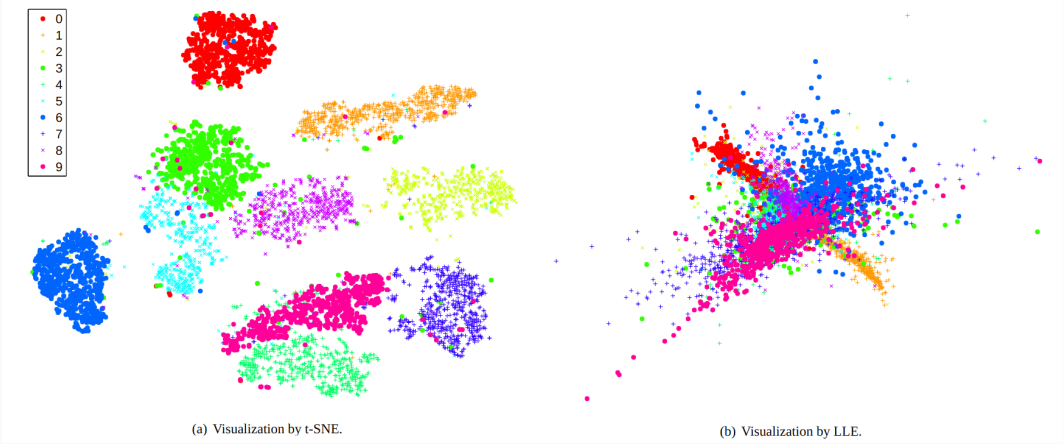



(a) Visualization by t-SNE.



(a) Visualization by Isomap.

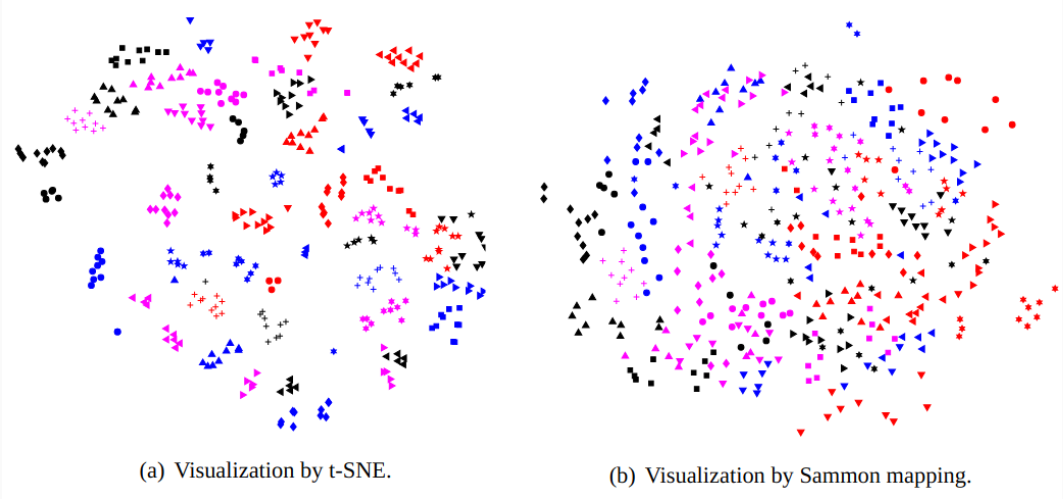
# Ejemplo: MNIST




(a) Visualization by t-SNE.

(b) Visualization by LLE.

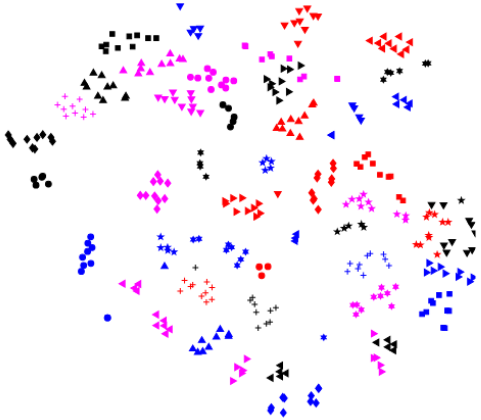
# Ejemplo: Olivetti



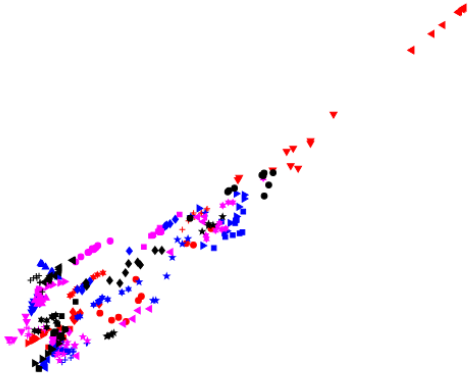
(a) Visualization by t-SNE.

(b) Visualization by Sammon mapping.

# Ejemplo: Olivetti

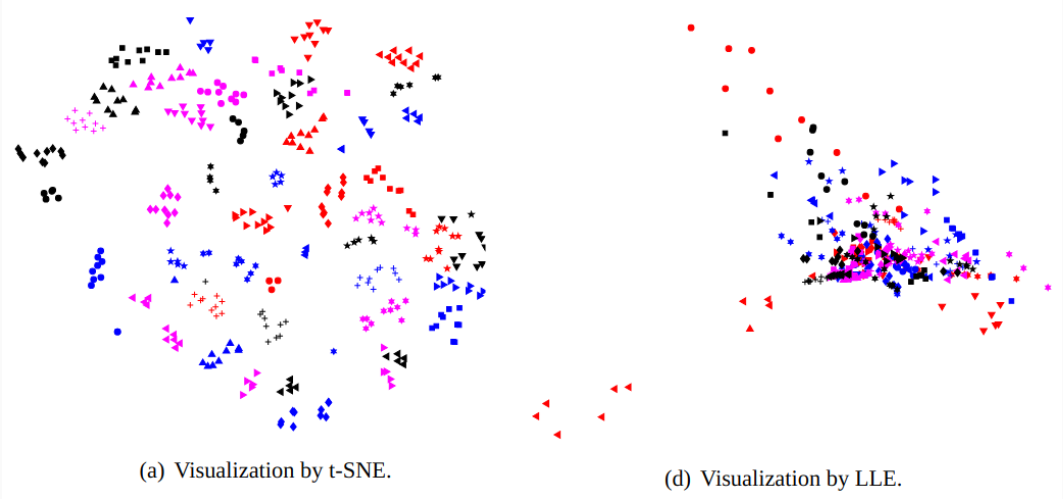


(a) Visualization by t-SNE.



(c) Visualization by Isomap.

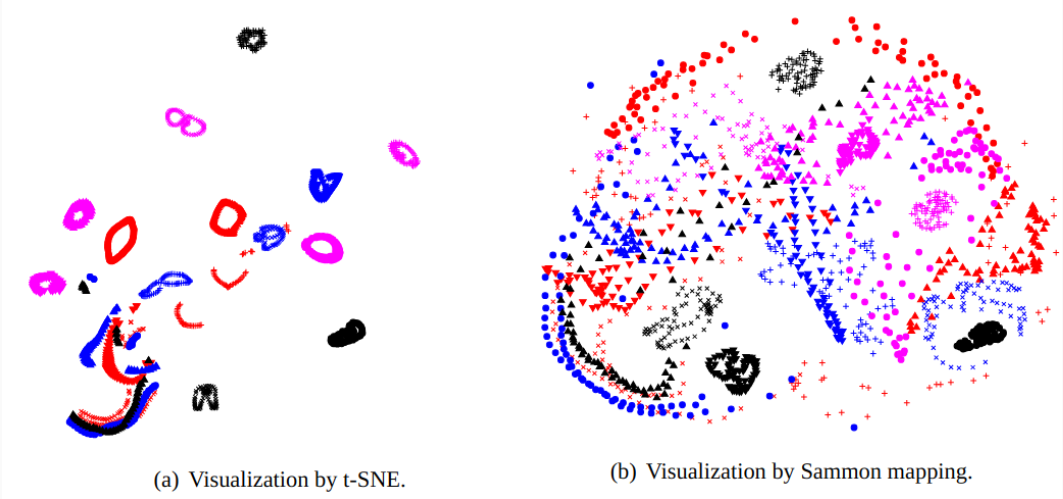
# Ejemplo: Olivetti



(a) Visualization by t-SNE.

(d) Visualization by LLE.

# Ejemplo: Coil-20



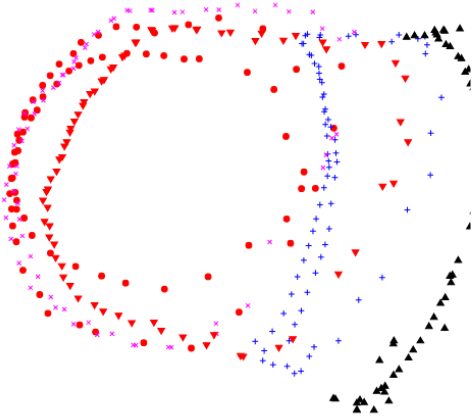
(a) Visualization by t-SNE.

(b) Visualization by Sammon mapping.

# Ejemplo: Coil-20

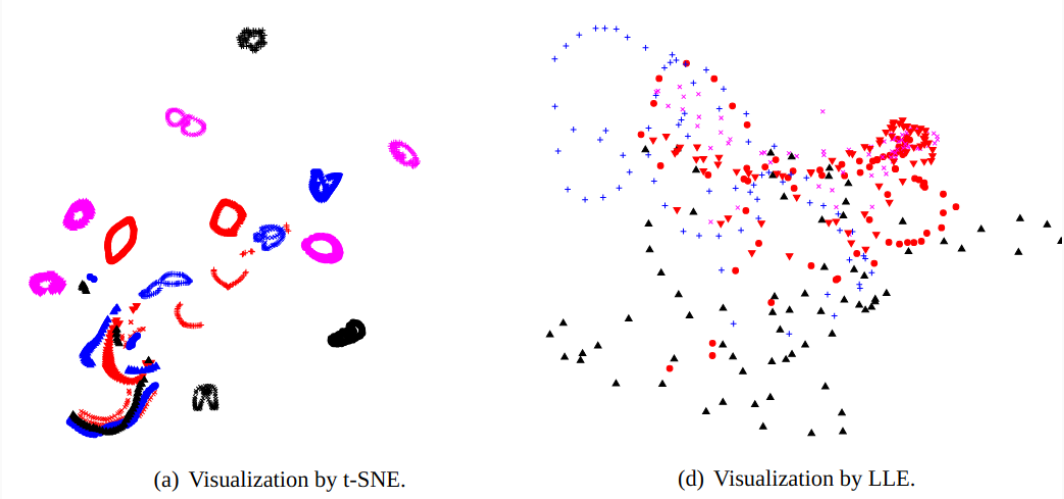


(a) Visualization by t-SNE.



(c) Visualization by Isomap.

# Ejemplo: Coil-20



(a) Visualization by t-SNE.

(d) Visualization by LLE.

# t-SNE en conjuntos grandes

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.
  - Comenzamos eligiendo un número deseado de vecinos y construyendo un grafo de vecindad.

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.
  - Comenzamos eligiendo un número deseado de vecinos y construyendo un grafo de vecindad.
  - Queda asi definido un subconjunto de puntos llamados **landmarks**.

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.
  - Comenzamos eligiendo un número deseado de vecinos y construyendo un grafo de vecindad.
  - Queda asi definido un subconjunto de puntos llamados **landmarks**.
  - Para cada uno de los puntos landmark, definimos un **random walk** que comienza en ese punto y termina cuando llega a otro landmark.

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.
  - Comenzamos eligiendo un número deseado de vecinos y construyendo un grafo de vecindad.
  - Queda asi definido un subconjunto de puntos llamados **landmarks**.
  - Para cada uno de los puntos landmark, definimos un **random walk** que comienza en ese punto y termina cuando llega a otro landmark.
- Ventaja:

# t-SNE en conjuntos grandes

- Complejidad $O(n^2)$ en tiempo y memoria.
- Para muchos puntos:
  - Necesitamos reducir la cantidad, pero sin perder estructura.
  - Comenzamos eligiendo un número deseado de vecinos y construyendo un grafo de vecindad.
  - Queda asi definido un subconjunto de puntos llamados **landmarks**.
  - Para cada uno de los puntos landmark, definimos un **random walk** que comienza en ese punto y termina cuando llega a otro landmark.
- Ventaja:
  - Se aprovecha la estructura de todos los datos.

# Limitaciones y debilidades

# Limitaciones y debilidades

1. No es claro como funciona en reducción a dimensiones más altas (más de 3D).

# Limitaciones y debilidades

1. No es claro como funciona en reducción a dimensiones más altas (más de 3D).
2. Maldición de la dimensionalidad (como todo el resto de métodos).

# Limitaciones y debilidades

1. No es claro como funciona en reducción a dimensiones más altas (más de 3D).
2. Maldición de la dimensionalidad (como todo el resto de métodos).
3. La función de coste no es convexa:

# Limitaciones y debilidades

1. No es claro como funciona en reducción a dimensiones más altas (más de 3D).
2. Maldición de la dimensionalidad (como todo el resto de métodos).
3. La función de coste no es convexa:
    - Hay que encontrar buenos valores para los hiperparámetros.

# Conclusiones

# Conclusiones

- t-SNE introduce:

# Conclusiones

- t-SNE introduce:
  - Nueva tecnica de visualización no lineal que mantiene la estructura local.

# Conclusiones

- t-SNE introduce:
  - Nueva tecnica de visualización no lineal que mantiene la estructura local.
  - Aunque es $O(n^2)$, existen soluciones para conjuntos grandes.

# Conclusiones

- t-SNE introduce:
  - Nueva tecnica de visualización no lineal que mantiene la estructura local.
  - Aunque es $O(n^2)$, existen soluciones para conjuntos grandes.
  - Funciona mucho mejor que técnicas previas en datos reales.

# Visualizing Data using t-SNE

Laurens van der Maaten & Geoffrey Hinton (2008)